Documentación Técnica: Arquitectura y Despliegue de KrisBot en Producción

Desarrollador: Kristhian Lizcano **Tecnologías:** Rasa, Python, Groq (Llama 3), Node.js, WhatsApp Web JS, PM2, Ubuntu Server.

1. Introducción

El presente documento detalla el proceso de desarrollo, depuración y despliegue en producción de **KrisBot**, un asistente virtual impulsado por Inteligencia Artificial generativa. El objetivo del proyecto fue construir una arquitectura robusta capaz de procesar lenguaje natural de forma dinámica y conectarse simultáneamente a múltiples plataformas de mensajería (Telegram y WhatsApp), operando de forma ininterrumpida (24/7) en un servidor Ubuntu en la nube.

A diferencia de los chatbots tradicionales basados en flujos rígidos, KrisBot utiliza Rasa como enrutador principal y delega la toma de decisiones complejas a un modelo de lenguaje de gran escala (LLM) a través de la API de Groq, logrando una latencia ultrabaja y respuestas conversacionales fluidas.

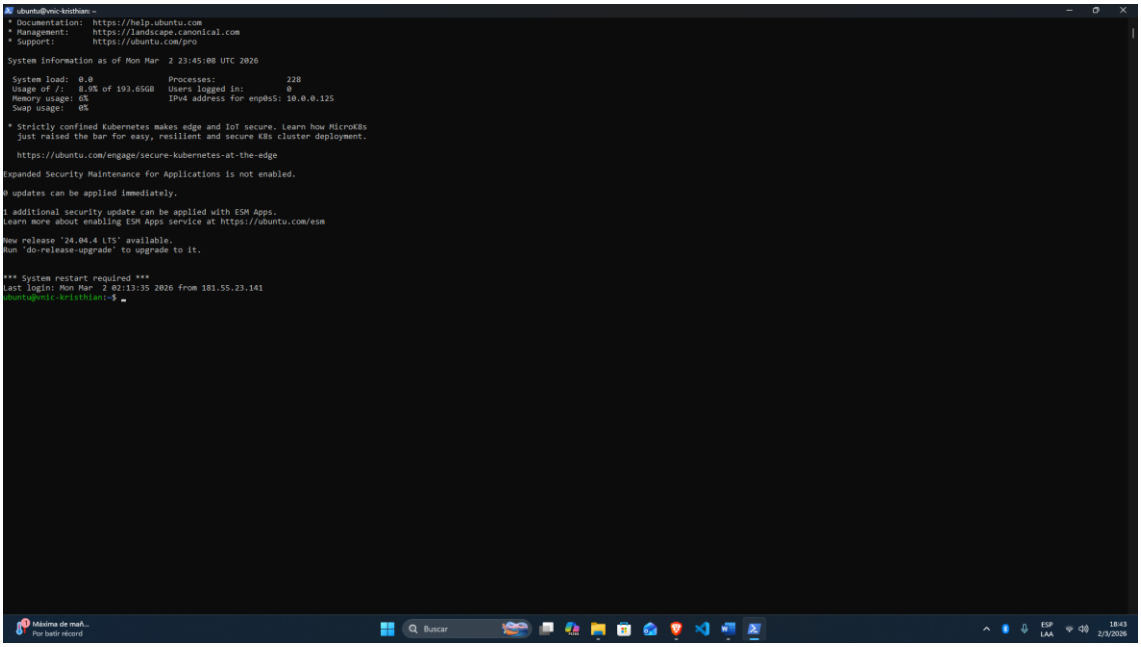
[Insertar captura de pantalla: Una conversación fluida con KrisBot en WhatsApp o Telegram]

---

2. Arquitectura del Sistema

La infraestructura del bot se divide en tres capas principales que se comunican entre sí:

1. **El Motor de Diálogo (Rasa Core - Puerto 5005):** Recibe los mensajes de los distintos canales, gestiona la memoria temporal del usuario y activa las reglas de negocio.
2. **El Cerebro (Action Server - Puerto 5055):** Un servidor en Python que recibe el contexto de Rasa, estructura un prompt dinámico y consulta la API de Groq para generar la respuesta.
3. **Los Puentes de Comunicación (Node.js & Python):** Scripts intermediarios que conectan las APIs de Telegram y WhatsApp Web con el puerto 5005 de Rasa. Todo mantenido en segundo plano mediante PM2 y nohup.



---

3. Retos Técnicos y Soluciones Implementadas

Durante la fase de despliegue en el servidor, se presentaron diversos desafíos de integración y configuración que requirieron depuración avanzada. A continuación, se documentan los principales retos superados:

Reto 1: El Bucle Infinito y el "Circuit Breaker" de Rasa

El Problema: Al recibir un mensaje del usuario que coincidía con intenciones predeterminadas (ej. mood_great_es), el motor predictivo de Rasa (TED Policy) entraba en un bucle infinito intentando adivinar el siguiente paso, lo que terminaba colapsando el servidor y activando el cortafuegos (Circuit breaker tripped). **La Solución:** Se reestructuró la arquitectura de Rasa para convertirlo en un enrutador estricto. Se eliminaron las intenciones innecesarias en domain.yml y nlu.yml, se vació el archivo stories.yml y se forzó todo el tráfico desconocido hacia la IA mediante una regla absoluta de nlu_fallback en rules.yml.

Reto 2: Fallo de Sintaxis en el Prompt (Python)

El Problema: El comportamiento de la IA era errático. Al revisar el código en actions.py, se descubrió que el uso de símbolos de numeral (#) dentro de un *f-string* de Python estaba siendo interpretado por la IA como formato Markdown (Títulos H1), desconfigurando las instrucciones de personalidad. **La Solución:** Se limpió el prompt, eliminando comentarios internos y ajustando la indentación estricta de Python para garantizar que la API de Groq recibiera un texto de sistema limpio.

Reto 3: Incompatibilidad del Entorno de Node.js

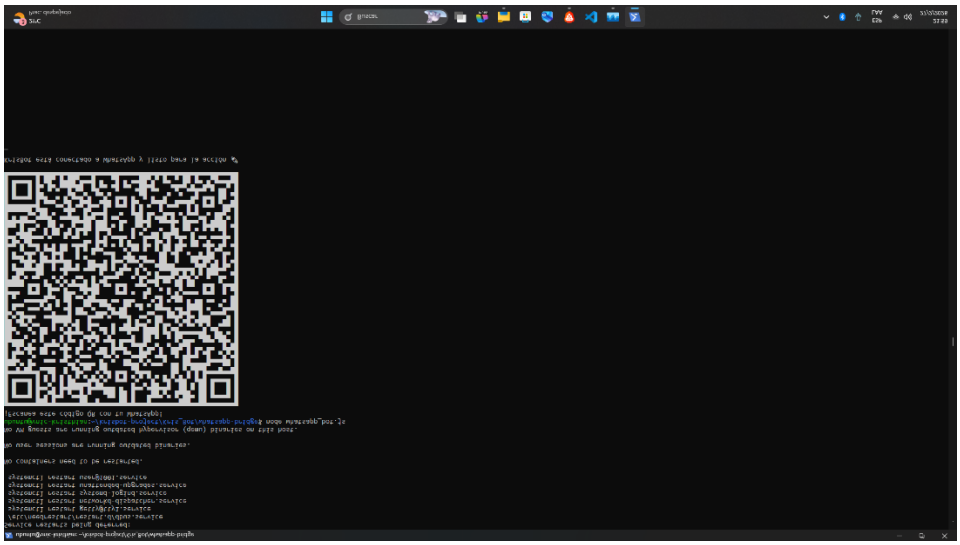
El Problema: Al iniciar el desarrollo del puente para WhatsApp usando la librería whatsapp-web.js, el instalador de NPM (npm install) falló arrojando múltiples errores de tipo EBADENGINE y SyntaxError. **La Solución:** Se diagnosticó que el servidor Ubuntu contaba con una versión obsoleta de Node.js (v12). Se purgó el sistema y se instaló Node.js v20 a través de NodeSource, actualizando el gestor de paquetes NPM a las exigencias de las librerías modernas.

Reto 4: Ausencia de Librerías Gráficas en Ubuntu (Headless Chrome)

El Problema: Al intentar ejecutar el script de Node.js para WhatsApp, el proceso explotó con el error error while loading shared libraries: libatk-1.0.so.0. **La Solución:** Dado que el servidor virtual (VPS) no posee interfaz gráfica, Chromium (usado por Puppeteer para escanear el QR) no podía arrancar. Se ejecutó un comando masivo en Ubuntu para instalar todas las dependencias gráficas necesarias (libx11-xcb1, libnss3, libgtk-3-0, etc.), permitiendo que el navegador operara de forma invisible.

Reto 5: La Lógica del Chat Propio en WhatsApp

El Problema: Una vez conectado a WhatsApp, el bot no respondía cuando el desarrollador se escribía mensajes a sí mismo para realizar pruebas, causando un "silencio de radio" en los logs. **La Solución:** Se comprendió el funcionamiento de la librería, la cual ignora los mensajes salientes por defecto. Para pruebas seguras, se implementó el evento message_create filtrado por la palabra clave /bot. Posteriormente, para la fase de producción (Bot Público), se utilizó una línea de WhatsApp Business dedicada y se ajustó el listener estándar (client.on('message')) para responder a todo tráfico entrante externo, protegiendo al bot de leer grupos o estados.



Reto 6: Gestión de Credenciales y el Error 401

El Problema: Durante las pruebas finales de integración, la consola de Python arrojó un error 401 - Invalid API Key. **La Solución:** Se detectó que la llave de la API de Groq había sido revocada. Se generó una nueva credencial en la consola del proveedor, se inyectó en el código fuente y se reinició el servidor de acciones de Rasa limpiando la memoria caché.

---

4. Inmortalidad del Sistema (Despliegue Continuo)

Para asegurar que KrisBot operara 24/7 sin depender de una sesión activa de SSH en la terminal, se implementó un sistema de persistencia de procesos:

- Para el backend en Python (Rasa Core y Action Server), se utilizó nohup desviando los errores a archivos .log dedicados.
- Para el puente de Node.js (WhatsApp), se configuró el gestor de procesos avanzados **PM2** (pm2 start, pm2 save, pm2 startup), garantizando que el bot se reinicie automáticamente ante cualquier fallo del servidor o mantenimiento en la nube.

5. Conclusión

La arquitectura actual de KrisBot demuestra la viabilidad de integrar motores conversacionales clásicos (Rasa) con modelos fundacionales modernos (Llama 3 via Groq) a través de scripts personalizados, logrando un ecosistema de mensajería autónomo, escalable y resiliente ante caídas. Links: https://t.me/Kristhbot y https://w.app/pdk8ot